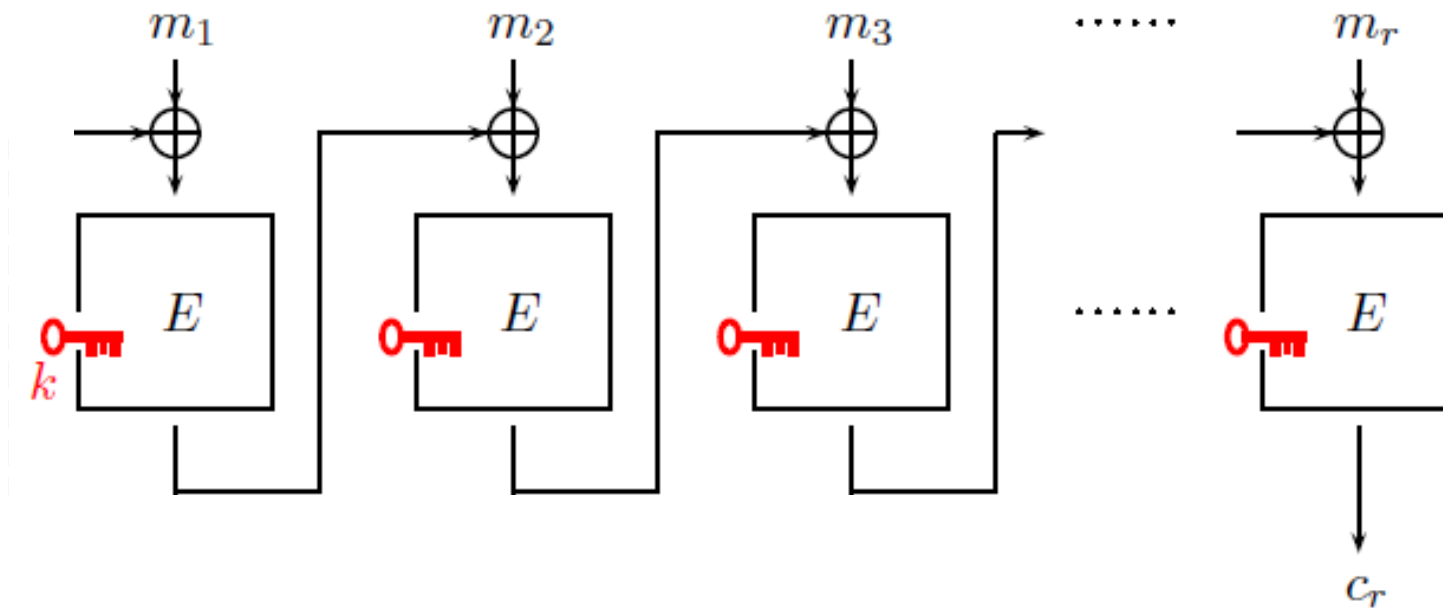


Dieser Abschnitt enthält einige MAC-Konstruktionen auf Basis von Blockverschlüsselungsverfahren, deren Blocklänge (in Bytes) mit n bezeichnet sei:

$$E : \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{M} \quad \text{mit} \quad \mathcal{M} := (\mathbb{Z}_2^8)^n$$



NIST Special Publication 800-38B - Recommendation for Block Cipher Modes of Operation: **The CMAC Mode for Authentication**

CMAC-Verfahren (Cipher-based MAC)

Das CMAC-Verfahren ist im NIST-Standard SP 800-38B (Recommendation for Block Cipher Modes of Operation: The CMAC Mode for Authentication) spezifiziert.

Gegeben sei ein Blockverschlüsselungsverfahren, deren Blocklänge (in Bytes) mit n bezeichnet sei:

$$E : \mathcal{K} \times (\mathbb{Z}_2^8)^n \rightarrow (\mathbb{Z}_2^8)^n$$

Für das MAC-Verfahren

$$M = \text{CMAC-}E : \mathcal{K} \times (\mathbb{Z}_2^8)^{\geq 0} \rightarrow (\mathbb{Z}_2^8)^n$$

ist die Berechnung des MAC-Werts

$$M_k(m) = M(k, m), \quad k \in \mathcal{K}, m \in (\mathbb{Z}_2^8)^{\geq 0}$$

für eine durch ein Bytearray m gegebene Nachricht folgendermaßen definiert.

$\mathbb{Z}_2^8$: Bytes
 n : Blocklänge
(AES: $n = 16$)

■ *Padding von m :*

m wird mit einem Bytearray der Form

80 00 ... 00

erweitert, sodass die Bytelänge des resultierenden Arrays ein Vielfaches der Blocklänge n ist. Bezeichnet $l(m)$ die Bytelänge von m , so gilt:

$$\tilde{m} := \begin{cases} m & \text{falls } l(m) \bmod n = 0 \text{ und } l(m) \neq 0 \\ m \parallel \underbrace{80\ 00\ \dots\ 00}_{n - (l(m) \bmod n)} & \text{falls } l(m) \bmod n \neq 0 \text{ oder } l(m) = 0 \end{cases}$$

■ *Zerlegung von $\tilde{m}$ in Blöcke:*

$$\tilde{m} = m_1 \parallel \dots \parallel m_{r-1} \parallel \tilde{m}_r \quad \text{mit } l(m_1) = \dots = l(m_{r-1}) = l(\tilde{m}_r) = n$$

■ *XOR-Verschlüsselung von $\tilde{m}_r$:*

$$m_r := \begin{cases} \tilde{m}_r \oplus k_1 & \text{falls } l(m) \bmod n = 0 \text{ und } l(m) \neq 0 \\ \tilde{m}_r \oplus k_2 & \text{falls } l(m) \bmod n \neq 0 \text{ oder } l(m) = 0 \end{cases}$$

Hierbei sind k_1 und k_2 Modifikationen des Blocks k_0 , der sich durch Verschlüsselung des Nullblocks ergibt:

$$k_0 := E_k(\mathbf{0}) = E_k(00\ 00 \dots 00)$$

Die durchzuführenden Operationen zur Modifikation von k_0 zu k_1 beziehungsweise k_2 können der Beispielimplementierung aus Abbildung 5.2 entnommen werden.

■ *CBC-Verschlüsselung von $(m_1, \dots, m_{r-1}, m_r)$ mit $IV = \mathbf{0}$:*

$$(c_1, c_2, \dots, c_r) := E_{k, \mathbf{0}}(m_1, \dots, m_r)$$

■ *Ausgabe des MAC-Werts:*

$$M_k(m) = c_r$$

NIST Special Publication 800-38B - Recommendation for Block Cipher Modes of Operation: The CMAC Mode for Authentication

6.1 Subkey Generation

Steps:

1. Let $L = \text{CIPH}_K(0^b)$.
2. If $\text{MSB}_1(L) = 0$, then $K1 = L \ll 1$;
Else $K1 = (L \ll 1) \oplus R_b$; see Sec. 5.3 for the definition of R_b .
3. If $\text{MSB}_1(K1) = 0$, then $K2 = K1 \ll 1$;
Else $K2 = (K1 \ll 1) \oplus R_b$.
4. Return $K1, K2$.

$$L = k_0 \Leftrightarrow k_0(x) = \sum_{i=0}^{127} a_i \cdot x^i$$

$$\left\{ \begin{array}{l} k_1 = x \cdot k_0(x) \bmod m(x) \\ k_2 = x \cdot k_1(x) = x^2 \cdot k_0(x) \bmod m(x) \end{array} \right.$$

$\Rightarrow$ Insbesondere sind $K1, K2$ verschieden

5.3 Subkeys

One of the elements of the subkey generation process is a bit string, denoted R_b , that is completely determined by the number of bits in a block. In particular, for the two block sizes of the currently approved block ciphers, $R_{128} = 0^{120}10000111$, and $R_{64} = 0^{59}11011$.

In general, R_b is a representation of a certain irreducible binary polynomial of degree b , namely, the lexicographically first among all such polynomials with the minimum possible number of nonzero terms. If this polynomial is expressed as $u^b + c_{b-1}u^{b-1} + \dots + c_2u^2 + c_1u + c_0$, where the coefficients $c_{b-1}, c_{b-2}, \dots, c_2, c_1, c_0$ are either 0 or 1, then R_b is the bit string $c_{b-1}c_{b-2} \dots c_2c_1c_0$.

R_{128} entspricht irred. Polynom

$$m(x) = x^{128} + x^7 + x^2 + x + 1 \in \mathbb{Z}_2[x]$$

$$\text{Def. Körper } \mathbb{F}_{2^{128}} = \{ p(x) \in \mathbb{Z}_2[x] \mid \text{grad}(p) \leq 127 \}$$

CBC-MAC-Verfahren

Besteht die Nachrichtenmenge nur aus Nachrichten einer festen Bytelänge, die ein Vielfaches b der Blocklänge n ist, so kann anstelle des CMAC-Verfahrens auch das einfachere CBC-MAC-Verfahren benutzt werden. Hierbei wird kein Padding eingesetzt und die XOR-Verknüpfung des letzten Blocks der Nachricht mit einem von k abgeleiteten Schlüssel entfällt:

$$M = \text{CBC-MAC-}E : \mathcal{K} \times (\mathbb{Z}_2^8)^{n \cdot b} \rightarrow (\mathbb{Z}_2^8)^n$$

$$M_k((m_1, \dots, m_b)) := c_b \quad \text{mit } (c_1, c_2, \dots, c_b) = E_{k,0}(m_1, \dots, m_b)$$

Warnung Essenziell für eine sichere Anwendung des CBC-MAC-Verfahrens ist die genannte Voraussetzung, dass alle Nachrichten die gleiche Länge besitzen, die zudem ein Vielfaches der Blocklänge sein muss.

Sind $m_1, \dots, m_r$ und $\tilde{m}_1, \dots, \tilde{m}_s$ Blöcke, so gilt für $M = \text{CBC-MAC-}E$ beispielsweise:

$$M_k(m) \stackrel{\ell(m)}{=} M_k \left((m, m \oplus M_k(m)) \right) \stackrel{2 \cdot \ell(m)}{}$$

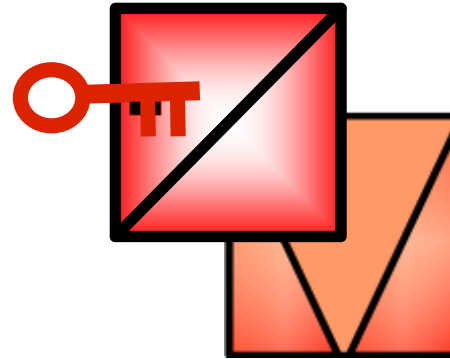
$$M_k((m_1, \dots, m_r, \mathbf{0}, \tilde{m}_1, \dots, \tilde{m}_s)) = M_k \left((M_k(m_1, \dots, m_r), \tilde{m}_1, \dots, \tilde{m}_s) \right)$$

$$\underbrace{\hspace{10em}}_{r+s+1} \downarrow$$

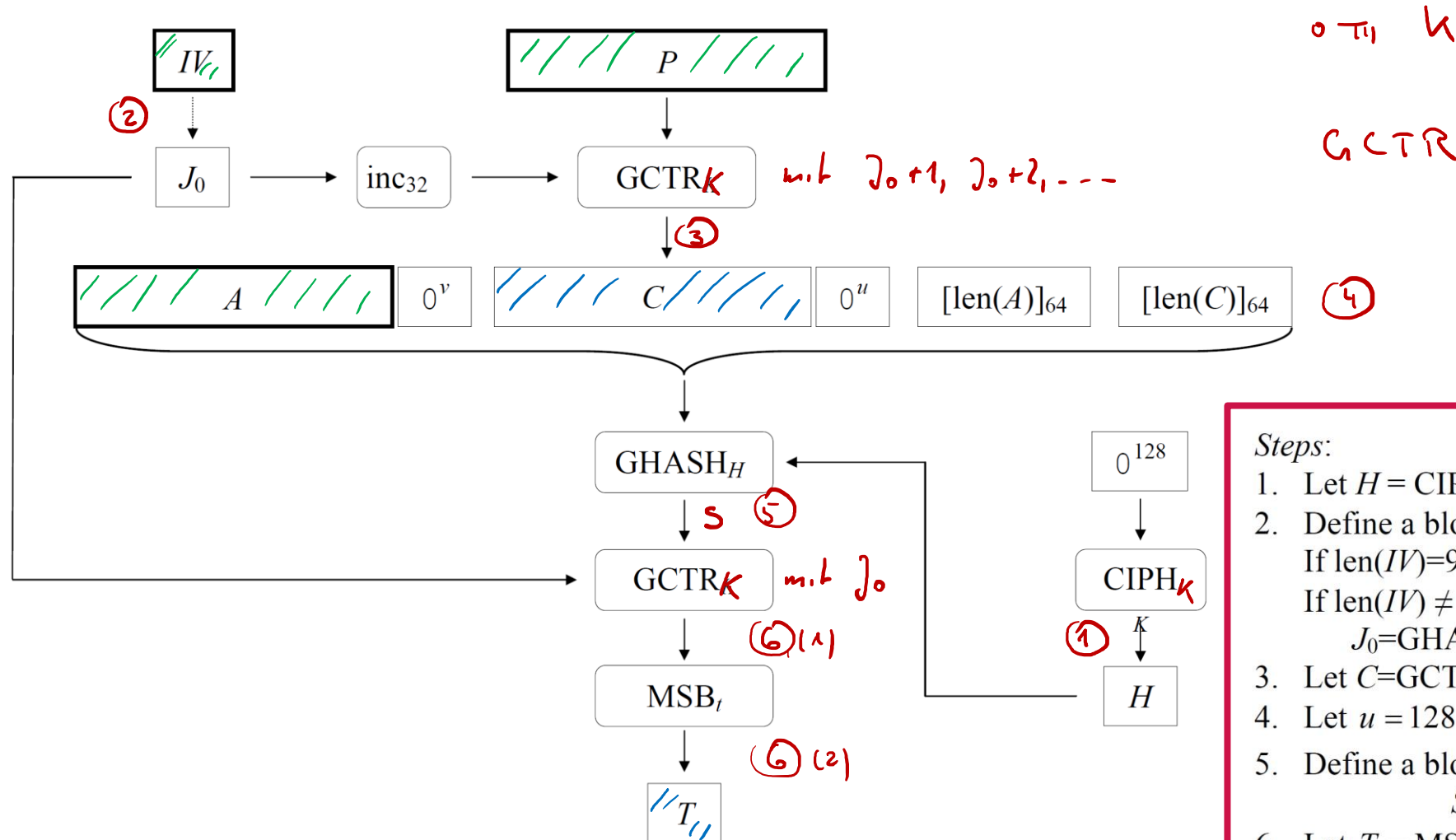
$$M_k(m_1, \dots, m_r)$$

$$\underbrace{\hspace{10em}}_{s+1}$$

- Authenticated Encryption with Associated Data



- Algorithm for combined encryption/decryption and MAC calculation/verification
- Encryption and MAC calculation:
 - Input: Plaintext P , Additional Data A , Key k , Nonce IV
 - Output: Ciphertext $C = E_{k,IV}(P)$, MAC $T = MAC_{k,IV}(A, C)$
- ***AEAD algorithm used with TLS:***
 - Galois/Counter Mode (GCM) of operation of the AES algorithm ([NIST Special Publication 800-38D](#))
 - ChaCha20 and Poly1305 ([RFC 8439](#))



$$GCTR_K = AES-CTR$$

Steps:

1. Let $H = CIPH_K(0^{128})$.
2. Define a block, J_0 , as follows:
If $\text{len}(IV) = 96$, then let $J_0 = IV \parallel 0^{31} \parallel 1$.
If $\text{len}(IV) \neq 96$, then let $s = 128 \lceil \text{len}(IV)/128 \rceil - \text{len}(IV)$, and let $J_0 = \text{GHASH}_H(IV \parallel 0^{s+64} \parallel [\text{len}(IV)]_{64})$.
3. Let $C = GCTR_K(\text{inc}_{32}(J_0), P)$.
4. Let $u = 128 \cdot \lceil \text{len}(C)/128 \rceil - \text{len}(C)$ and let $v = 128 \cdot \lceil \text{len}(A)/128 \rceil - \text{len}(A)$.
5. Define a block, S , as follows:
$$S = \text{GHASH}_H(A \parallel 0^v \parallel C \parallel 0^u \parallel [\text{len}(A)]_{64} \parallel [\text{len}(C)]_{64})$$
6. Let $T = \text{MSB}_t(GCTR_K(J_0, S))$.
7. Return (C, T) .

Figure 3: $GCM-AE_K(IV, P, A) = (C, T)$.

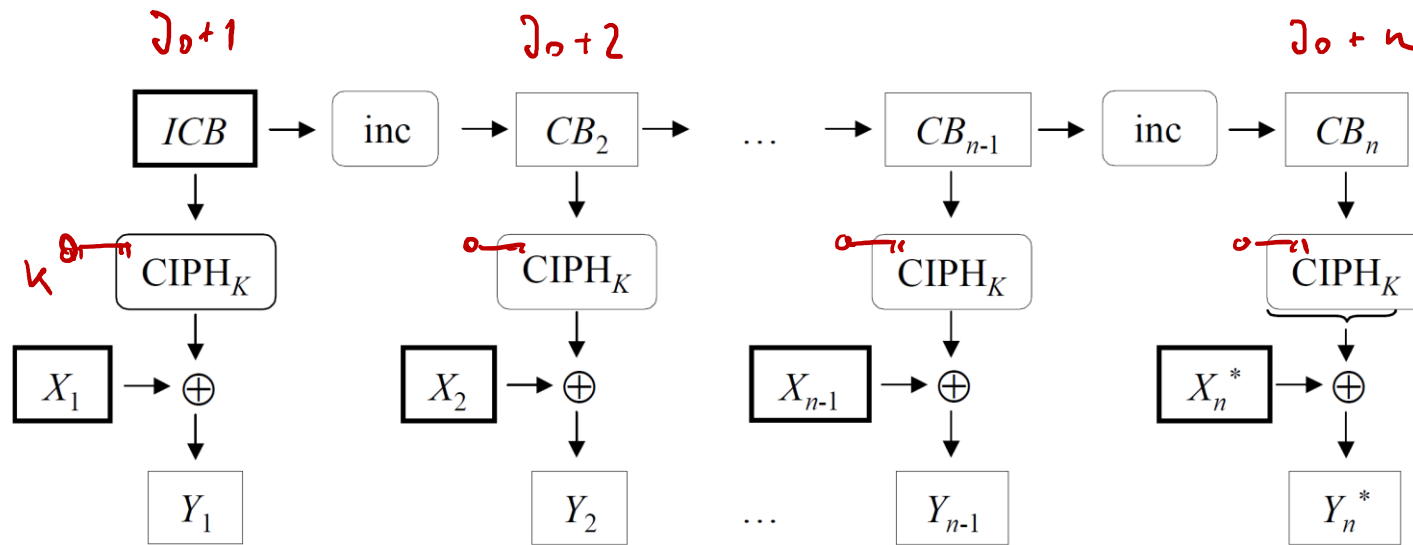


Figure 2: $GCTR_K(ICB, X_1 \parallel X_2 \parallel \dots \parallel X_n^*) = Y_1 \parallel Y_2 \parallel \dots \parallel Y_n^*$.

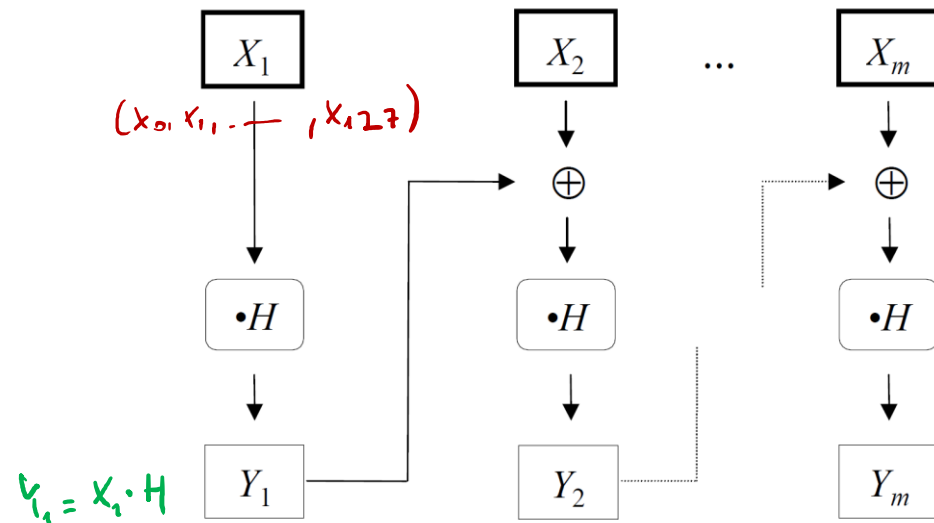


Figure 1: $\text{GHASH}_H(X_1 \parallel X_2 \parallel \dots \parallel X_m) = Y_m$.

6.3 Multiplication Operation on Blocks

Let R be the bit string $11100001 \parallel 0^{120}$. Given two blocks X and Y , Algorithm 1 below computes a “product” block, denoted $X \cdot Y$:

Algorithm 1: $X \cdot Y$

Steps:

1. Let $x_0x_1\dots x_{127}$ denote the sequence of bits in X .
2. Let $Z_0 = 0^{128}$ and $V_0 = Y$.
3. For $i = 0$ to 127 , calculate blocks Z_{i+1} and V_{i+1} as follows:

$$Z_{i+1} = \begin{cases} Z_i & \text{if } x_i = 0; \\ Z_i \oplus V_i & \text{if } x_i = 1. \end{cases}$$

$$V_{i+1} = \begin{cases} V_i \gg 1 & \text{if } \text{LSB}_1(V_i) = 0; \\ (V_i \gg 1) \oplus R & \text{if } \text{LSB}_1(V_i) = 1. \end{cases}$$

4. Return Z_{128} .

$V_0 \leftarrow P_Y(x)$

$V_i \leftarrow P_Y(x) \cdot x^i \bmod P_R(x)$

The $\cdot$ operation on (pairs of) the 2^{128} possible blocks corresponds to the multiplication operation for the binary Galois (finite) field of 2^{128} elements. The fixed block, R , determines a representation of this field as the modular multiplication of binary polynomials of degree less than 128.

$$X \cdot Y \leftrightarrow \left(\sum_{i=0}^{127} P_Y(x) \cdot x^i \cdot x^i \right) \bmod P_R(x) = \sum_{i=0}^{127} (P_Y \cdot x_i \cdot x^i) \bmod P_R(x)$$

$$Z_i \leftarrow \sum_{j=0}^{i-1} P_Y \cdot x_j \cdot x^j \quad (i \geq 1)$$

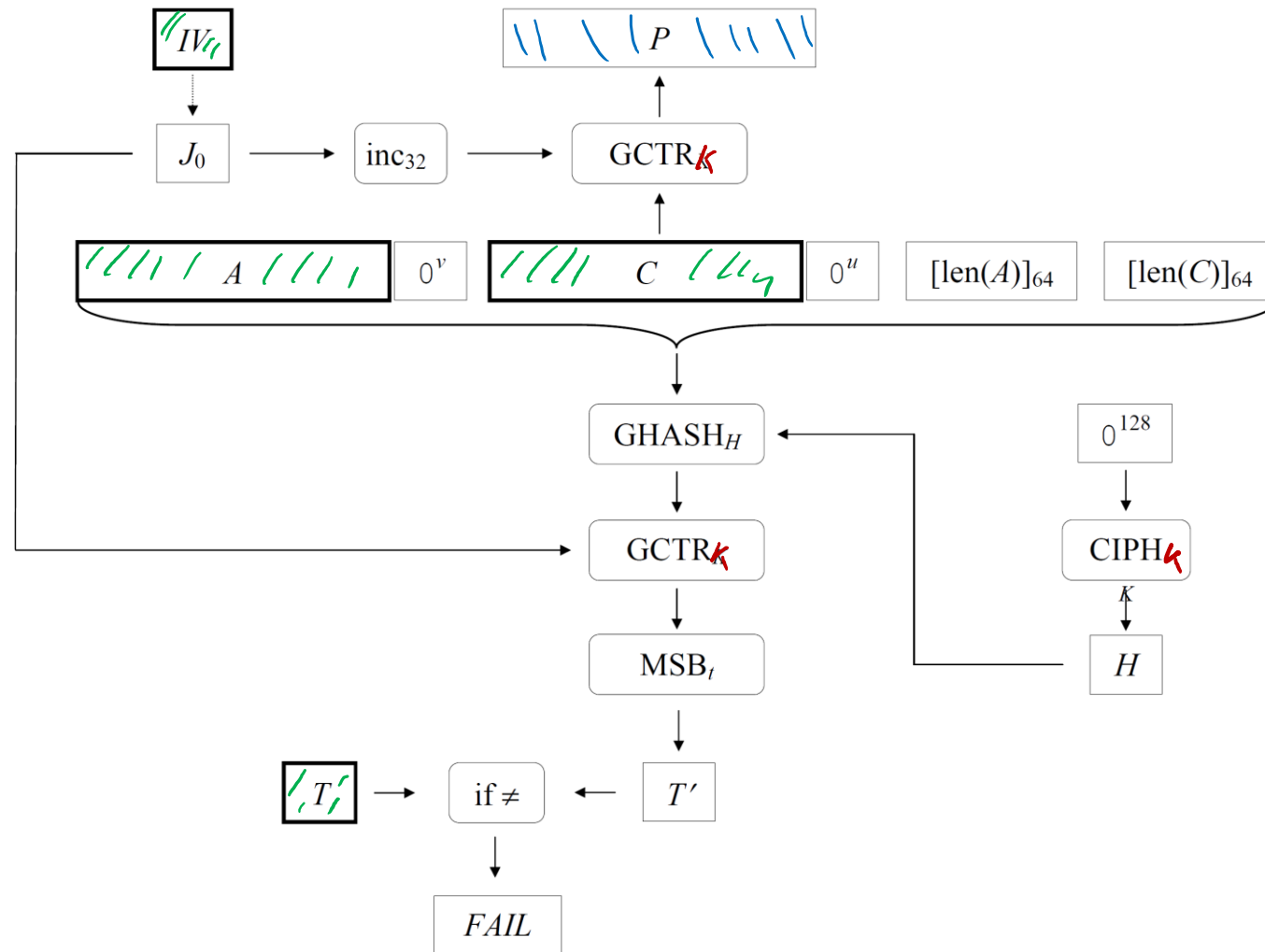


Figure 4: $GCM-AD_K(IV, C, A, T) = P$ or $FAIL$.

- ✓ ■ **Kryptographisch sichere Pseudozufallszahlengeneratoren**
Cryptographic Secure Pseudo Random Number Generators (CSPRNG)
- ✓ ■ **Kryptographische Hashfunktionen**
Message Digests (Cryptographic Hash Functions)
- ✓ ■ **Symmetrische Verschlüsselungsverfahren** (symmetric ciphers)
- ✓ ■ **Message Authentication Codes (MACs)**
- **Asymmetrische Verschlüsselungsverfahren** (asymmetric ciphers)
- **Digitale Signaturen** (digital signatures)
- **Schlüsselaustauschverfahren** (key exchange / key establishment)

- Public Key Cryptography

DH ■ Schlüsselaustauschverfahren,

RSA ■ asymmetrische Verschlüsselungsverfahren

RSA ■ Signaturverfahren.

DSA (Digital Signature Algo.)

EC-DSA



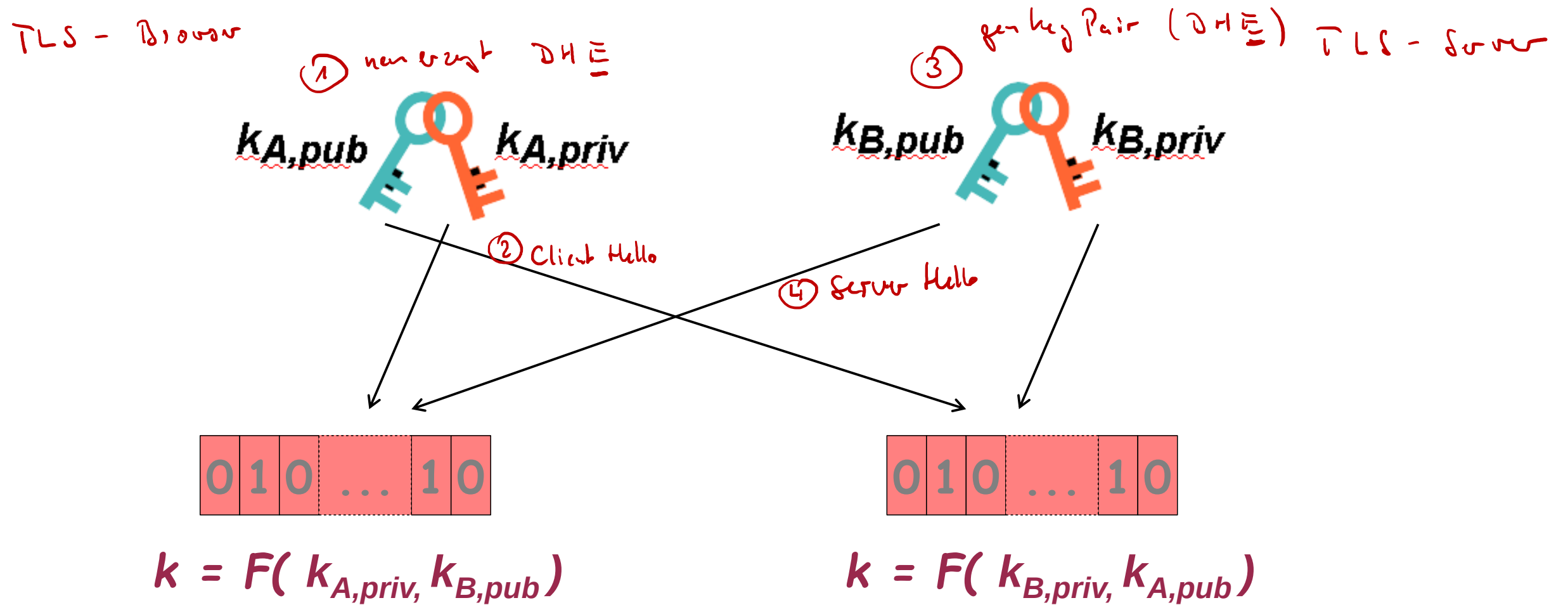
PQ-Algorithmen

modern
EC-DH
Elliptic
Curve - DH

1	Modulare Arithmetik - Teil 2	1
1.1	Potenzen	2
1.2	Mathematischer Exkurs: Einheitengruppen $\mathbb{Z}_{p^e}^*$	8
1.3	Der chinesische Restsatz	11
1.4	Mathematischer Exkurs: Elemente ungerader und gerader Ordnung in $\mathbb{Z}_n$	17
2	Das <u>Diffie-Hellman-Schlüsselaustauschverfahren</u> (klassisch) mit $\mathbb{Z}_n$	23
2.1	Exkurs: Berechnung modularer Potenzen	30
2.2	Exkurs: Angriffe auf das DH-Verfahren	34
2.2.1	Pohlig-Hellman-Reduktion	34
2.2.2	Shanks' „Baby Steps Giant Steps“-Verfahren (BSGS-Verfahren)	35
2.2.3	Pollard's <i>Rho-Methode</i>	35
2.2.4	Zahlkörpersieb	37
3	Das RSA-Verfahren	39
3.1	RSA-Schlüsselpaare	41
3.2	Das RSA-Verschlüsselungsverfahren	45
3.3	Das RSA-Signaturverfahren	51

Das Diffie-Hellman-Schlüsselaustauschverfahren

2.1	Exkurs: Berechnung modularer Potenzen	30
2.2	Exkurs: Angriffe auf das DH-Verfahren	34



Für Schlüsselpaare, die in asymmetrischen kryptographischen Verfahren zum Einsatz kommen, sollten folgende grundlegende Eigenschaften gelten:

- Ein Schlüsselpaar ist genau einem/einer „Besitzer/in“ zugeordnet. Alice besitzt beispielsweise das Schlüsselpaar:

$$k_A = (k_{A,priv}, k_{A,pub})$$

- Niemals hat jemand anderes als Alice Zugriff auf $k_{A,priv}$.



- Konsequenterweise muss Alice $k_{A,priv}$ selbständig generieren.
- Unter Verwendung von $k_{A,priv}$ berechnet Alice auch $k_{A,pub}$ mithilfe eines effizient ausführbaren Algorithmus.
- Es gibt keinen effizient ausführbaren Algorithmus mit dem $k_{A,priv}$ bestimmt werden kann, falls nur $k_{A,pub}$ bekannt ist.

Diffie-Hellman-Verfahren (DH-Verfahren)

- Alice und Bob vereinbaren zunächst zwei Parameter, eine große natürliche Zahl n und eine Zahl:

$$g \in \mathbb{Z}_n^*$$

Die Ordnung $o(g)$ von g muss hierbei mindestens durch eine sehr große Primzahl geteilt werden.

Alle weiteren Rechnungen werden in $\mathbb{Z}_n$ ausgeführt.

(Häufig nutzen Alice und Bob DH-Parameter n und g , die in einem Standard speziell zur Nutzung mit dem DH-Verfahren spezifiziert sind.)

- Alice bestimmt eine zufällige Zahl $e_A \in \mathbb{Z}_n^*$ als privaten Schlüssel, d. h.:

$$k_{A,priv} = e_A$$

und berechnet dann den öffentlichen Schlüssel in $\mathbb{Z}_n$:

$$k_{A,pub} = g^{e_A} \in \mathbb{Z}_n$$

Das Schlüsselpaar von Alice ist somit:

$$(k_{A,priv}, k_{A,pub}) = (e_A, g^{e_A})$$

- Bob bestimmt ebenso eine zufällige Zahl $e_B \in \mathbb{Z}_n^*$ und damit das Schlüsselpaar:

$$(k_{B,priv}, k_{B,pub}) = (e_B, g^{e_B})$$

- Alice und Bob tauschen ihre öffentlichen Schlüssel aus, d. h. Alice übermittelt g^{e_A} an Bob und Bob sendet g^{e_B} an Alice.

- Alice berechnet in $\mathbb{Z}_n$:

$$s_A := (g^{e_B})^{e_A}$$

Bob berechnet in $\mathbb{Z}_n$:

$$s_B := (g^{e_A})^{e_B}$$

$$= g^{e_A \cdot e_B}$$

- Es gilt

$$s_A = (g^{e_B})^{e_A} = g^{e_B \cdot e_A} = g^{e_A \cdot e_B} = (g^{e_A})^{e_B} = s_B$$

und $s_A = s_B$ ist der zwischen Alice und Bob vereinbarte Schlüssel, beziehungsweise ein sogenanntes *shared secret* aus dem weitere Schlüssel zur Verwendung in symmetrischen kryptographischen Verfahren geeignet extrahiert beziehungsweise abgeleitet werden.

